Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH



Instytut Telekomunikacji

Praca dyplomowa magisterska

na kierunku Inżynieria Internetu Rzeczy

Projekt i implementacja asystenta głosowego wykorzystującego lokalnie działające modele LLM na platformie sprzętowej o ograniczonych zasobach.

Krzysztof Kulma
Numer albumu 318476

promotor
dr inż. Daniel Paczesny

WARSZAWA 2026

Projekt i implementacja asystenta głosowego wykorzystującego lokalnie działające modele LLM na platformie sprzętowej o ograniczonych zasobach.

Streszczenie. Celem pracy było zwiększenie interaktywności wczasowiczów przebywających w domkach wypoczynkowych Ośrodka Wypoczynkowego Politechniki Warszawskiej w Wildze. Realizowano to przez wdrożenie lokalnego asystenta głosowego, który umożliwia wczasowiczom dostęp do informacji zbieranych przez sieć czujników IoT. Asystent ma w przystępny sposób tłumaczyć odczyty dotyczące warunków w domku i zużycia energii, bez konieczności korzystania z interfejsu technicznego.

Część implementacyjna polegała na zaprojektowaniu i uruchomieniu kompletnego łańcucha interakcji głosowej. Obejmuje on detekcję mowy, transkrypcję, dynamiczne budowanie kontekstu z danych IoT, lokalną generację odpowiedzi oraz syntezę mowy. System oparto na architekturze dwumodelowej. Większy model Nauczyciel asynchronicznie analizuje dane z sieci czujników, a mniejszy model Uczeń odpowiada na pytania użytkownika na urządzeniu brzegowym.

Część badawcza polegała na empirycznym doborze małego modelu językowego oraz parametrów jego pracy na sprzęcie o ograniczonych zasobach. Na Raspberry Pi 5 porównano kilka modeli. Zbadano wpływ kwantyzacji wag, hiperparametrów generacji oraz konstrukcji promptu na jakość odpowiedzi w języku polskim i na czas ich uzyskania.

Wykonana implementacja i przeprowadzone testy na urządzeniu o małych zasobach, jakim jest Raspberry Pi 5, pozwoliły wykazać, że użyteczny asystent głosowy może działać lokalnie, bez wysyłania zapytań do chmury. Najlepsza zbadana konfiguracja, model `gemma3:4b` poddany kwantyzacji Q4_K_M, z ponownym użyciem stałego prefiksu promptu, osiągnęła medianę czasu odpowiedzi (E2E, od zapytania do ostatniego tokenu) poniżej 10 s przy wskaźniku halucynacji 1,17%. Wyniki pokazują, że o jakości odpowiedzi decyduje przede wszystkim wybór modelu, a skrócenie opóźnienia wymaga utrzymania modelu w pamięci urządzenia i obniżenia precyzji wag.

Słowa kluczowe: asystent głosowy, LLM, IoT, smart home, inferencja lokalna

Design and implementation of a voice assistant using locally running LLM models on a resource-constrained hardware platform.

Abstract. The aim of this thesis was to increase the interactivity of guests staying at the Warsaw University of Technology holiday cottages in Wilga. This was done by deploying a local voice assistant that gives guests access to information collected by an IoT sensor network. The assistant explains cottage conditions and energy use in everyday language, so there is no need for a technical interface.

The implementation work designed and ran a complete voice interaction pipeline. It covers speech detection, transcription, dynamic context building from IoT data, local response generation, and speech synthesis. The system uses a dual-model architecture. A larger Teacher model analyses data from the sensor network in the background, and a smaller Student model answers the user on the edge device.

The research work selected a small language model and its operating parameters empirically, on resource-constrained hardware. Several models were compared on a Raspberry Pi 5. The study examined how weight quantization, generation hyperparameters, and prompt design affect the quality of answers in Polish and the time needed to produce them.

The implementation and the tests on a low-resource device, the Raspberry Pi 5, showed that a useful voice assistant can run locally, without sending queries to the cloud. The best configuration studied, the `gemma3:4b` model quantized to Q4_K_M, with reuse of a fixed prompt prefix, reached a median end-to-end response time, from query to last token, below 10 s, at a hallucination rate of 1.17%. The results show that answer quality depends primarily on the choice of model, while reducing latency requires keeping the model in device memory and lowering the precision of its weights.

Keywords: voice assistant, LLM, IoT, smart home, on-device inference

Spis treści

1. Wstęp . . . 9
1.1. Tło problemu . . . 9
1.2. Teza badawcza i cel pracy . . . 9
1.3. Zakres pracy . . . 10
2. Przegląd literatury i analiza rozwiązań . . . 11
2.1. Małe modele językowe . . . 11
2.1.1. Modele językowe . . . 11
2.1.2. Wielomodelowe systemy LLM . . . 11
2.1.3. Inżynieria promptów i dostarczanie kontekstu . . . 12
2.1.4. Optymalizacja hiperparametrów generacji . . . 12
2.1.5. Fine tuning modeli językowych . . . 12
2.2. Inferencja na urządzeniu brzegowym . . . 12
2.2.1. Architektury hybrydowe Edge–Cloud . . . 12
2.2.2. Kwantyzacja dla inferencji na sprzęcie bez GPU . . . 13
2.3. Interfejs głosowy . . . 13
2.3.1. Interfejsy głosowe: rozpoznawanie mowy, detekcja aktywności głosowej i synteza mowy . . . 13
3. Koncepcja i architektura systemu . . . 14
3.1. Założenia projektowe . . . 14
3.2. Architektura Nauczyciel–Uczeń . . . 14
3.3. Schemat przepływu danych . . . 14
4. Środowisko projektowo-badawcze . . . 15
4.1. Wybór platformy brzegowej . . . 15
4.2. Architektura sprzętowa . . . 15
4.3. Modele LLM w przetwarzaniu brzegowym danych . . . 15
4.4. Wykorzystane oprogramowanie i technologie . . . 15
4.4.1. Aplikacja użytkownika . . . 15
4.4.2. Warstwa serwerowa i komunikacyjna . . . 15
4.4.3. Moduły STT i TTS . . . 15
4.4.4. LLM . . . 15
4.4.5. Moduł kontekstu . . . 15
4.4.6. Narzędzia badawcze . . . 16
5. Część badawcza: analiza porównawcza i optymalizacja SLM na urządzeniu brzegowym . . . 17
5.1. Architektura i metodologia systemu testowego . . . 17
5.1.1. Komponenty systemu testowego . . . 17
5.1.2. Zbiór danych testowych . . . 17
5.1.3. Metryki oceny . . . 17
5.1.4. Dostrajanie parametrów pracy modelu . . . 17
5.1.5. Porównywane modele . . . 17

5.2. Etap Optuna 17
5.2.1. Warunek termiczny pomiaru 17
5.2.2. Dominacja flagi halucynacji 17
5.2.3. Opóźnienie E2E i model Gaussa 18
5.3. Etap kwantyzacji 18
5.3.1. Typy kwantyzacji GGUF 18
5.3.2. Wstępne pomiary E2E 18
5.3.3. Wyniki 18
5.3.4. Anomalie i wnioski 18
5.4. Inżynieria promptów i buforowanie prefiksu 18
5.4.1. Buforowanie prefiksu promptu systemowego 18
5.4.2. Budowa promptu systemowego 18
5.5. Rekomendacje produkcyjne 18
6. Część implementacyjna: realizacja prototypu 19
6.1. Architektura prototypu 19
6.2. Elementy systemu i wykorzystane technologie 19
6.2.1. Aplikacja użytkownika 19
6.2.2. Transkrypcja mowy (STT) 19
6.2.3. Moduł kontekstu 19
6.2.4. LLM Nauczyciel 19
6.2.5. SLM Uczeń 19
6.2.6. Synteza mowy (TTS) 19
6.2.7. Konfiguracja 19
6.3. Testy rozwiązania 20
7. Podsumowanie 21
7.1. Wnioski z przeprowadzonych badań 21
7.2. Ocena prototypu względem założeń 21
7.3. Plan rozwoju 21
Bibliografia 23

1. Wstęp

1.1. Tło problemu

W ramach projektu Wilga MiLL Uczelnia prowadziła badania w domkach wczasowych na terenie Ośrodka Wypoczynkowego Politechniki Warszawskiej w Wildze. W domkach testowano system IoT monitorujący zachowania użytkowników. Celem projektu było zbudowanie inteligentnej przestrzeni edukacyjnej typu *living lab* nakierowanej na edukację prośrodowiskową. W ramach badań w jednym z domków zaimplementowano sieć czujnikową IoT. W niniejszej pracy wykorzystano zebrane przez nią dane archiwalne.

System czujników generował znaczne ilości surowych danych. Choć były one bezcenne z perspektywy badawczej, pozostawały trudne do interpretacji dla przeciętnej osoby odwiedzającej Ośrodek. Nadmiar informacji oraz brak intuicyjnego interfejsu prowadzi do zmarnowania potencjału systemu. Utrudniało to realizację celu projektu, jakim było zwiększenie świadomości zużycia energii i wpływu człowieka na środowisko. Problem braku zainteresowania systemem IoT wśród wczasowiczów był już wcześniej przedmiotem badań zespołu projektowego. W ramach projektu INOT opracowano aplikację komunikująca się poprzez interfejs wizualny. Aplikacja wprowadziła postać Wilgusia, ptaka gospodarza domu. Wilguś miał przedstawiać dane w zrozumiały sposób i zwiększać atrakcyjność systemu.

Istotnym elementem tła problemu były również zbadane zachowania użytkowników. Dotychczasowe doświadczenia zespołu projektowego wskazywały na ambiwalentny stosunek wczasowiczów do obecności technologii w domkach. Niektórzy postrzegali urządzenia IoT jako zbędne lub naruszające ich prywatność. Przejawiało się to poprzez odłączania urządzeń od zasilania i dezorganizowało pracę systemu. Doświadczenia te wskazywały na potrzebę zwiększenia atrakcyjności systemu. Wczasowicze powinni mieć poczucie aktywnego korzystania z niego, a nie bycia obserwowanymi przez czujniki IoT.

1.2. Teza badawcza i cel pracy

Przyjęto założenie, że istniejący interfejs wizualny należy rozbudować o interfejs głosowy. Interakcja z nim powinna być naturalna, płynna i intuicyjna. Nie powinna być dla wczasowicza obciążeniem, lecz atrakcyjnym elementem pobytu w Wildze. System powinien w subtelny sposób dostarczać informacji interesujących także dla osoby nietechnicznej. Miało to zachęcać wczasowiczów do interakcji z inteligentnym otoczeniem zamiast skłaniać ich do wyłączania urządzeń IoT.

Postawiono tezę, mówiącą że na urządzeniu IoT o ograniczonych zasobach można uruchomić mały model językowy (SLM). Model ten może stanowić rdzeń responsywnego asystenta głosowego korzystającego z danych systemowych. Model działający na Raspberry Pi 5 powinien być wspierany przez moduły wdrożone poza urządzeniem brzegowym. Obejmowały one rozpoznawanie i syntezę mowy, dynamiczne budowanie kontekstu oraz asynchroniczną analizę danych. Przyjęty podział zadań miał umożliwić udzielanie rzeczowych odpowiedzi w języku polskim przy minimalizacji obciążenia urządzenia brzegowego.

Weryfikacja tezy obejmowała pomiar czasu odpowiedzi, ocenę zgodności odpowiedzi z dostarczonymi danymi oraz analizę występowania halucynacji. Zbadano również wpływ doboru modelu, parametrów generacji, kwantyzacji wag i konstrukcji promptu na jakość oraz szybkość działania systemu.

Głównym celem pracy było opracowanie, wdrożenie i przetestowanie asystenta głosowego. Miał on ułatwiać wczasowiczom zrozumienie danych zbieranych przez system IoT. Cele cząstkowe obejmowały zaprojektowanie architektury systemu oraz dobór i optymalizację małego modelu językowego do pracy na Raspberry Pi 5. Prace obejmowały również opracowanie mechanizmu dynamicznego budowania kontekstu, integrację pełnego łańcucha interakcji głosowej oraz ocenę jakości i czasu odpowiedzi. W asystencie zastosowano otwartoźródłowy model LLM.

Zakładanym rezultatem prac było przedstawienie technicznych danych gromadzonych w domku w przystępnej formie głosowej. Ocena wpływu asystenta na zainteresowanie oraz odbiór systemu przez wczasowiczów wymaga jednak odrębnych badań z udziałem użytkowników. Badania te nie stanowiły przedmiotu niniejszej pracy.

1.3. Zakres pracy

Prace rozpoczęto od etapu koncepcyjnego. Najpierw poszerzono wiedzę w obszarach związanych z tematem pracy. W rozdziale 2 przedstawiono przegląd literatury dotyczącej modeli językowych, architektur brzegowych i chmurowych, kwantyzacji modeli, systemów wielomodelowych, inżynierii promptów, optymalizacji parametrów generacji oraz interfejsów głosowych.

Na tej podstawie opracowano koncepcję architektury systemu oraz sposobu wdrożenia komponentów. W rozdziale 3 przedstawiono podział zadań między modułami systemu oraz przepływ danych między nimi. Następnie przygotowano środowisko projektowo-badawcze. W niniejszym rozdziale opisano również platformę Raspberry Pi 5, architekturę sprzętową oraz wykorzystane oprogramowanie otwartoźródłowe.

Po zbudowaniu koncepcji i środowiska dalsze prace podzielono na część badawczą oraz część implementacyjną. Fazę badawczą opisano w rozdziale 5. Obejmowała ona dobór i optymalizację konfiguracji modelu językowego uruchamianego na platformie o ograniczonych zasobach. Analizie poddano wpływ parametrów uruchomieniowych, wielkości modelu, kwantyzacji wag, zawartości promptu oraz warunków pracy urządzenia na jakość i czas generowania odpowiedzi.

Część implementacyjna objęła wdrożenie asystenta głosowego zgodnie ze zbudowaną koncepcją oraz rekomendacjami wynikającymi z badań. Opis tej fazy przedstawiono w rozdziale 6. Prace skupiły się na zbudowaniu modułów systemu, a następnie ich integracji. Zbudowano w ten sposób spójny przepływ danych: od zapytania głosowego użytkownika, przez transkrypcję i budowanie kontekstu, po lokalną inferencję modelu Ucznia i syntezę odpowiedzi. Ponieważ podczas prac dane z Wilgi nie były dostępne na żywo, przygotowano warstwę danych zbudowaną w oparciu o zapisy archiwalne.

2. Przegląd literatury i analiza rozwiązań

Pracę nad rozwiązaniem poprzedzono przeglądem literatury oraz dostępnych rozwiązań. Przetwarzanie języka naturalnego (ang. *Natural Language Processing*, NLP) jest dziś zdominowane przez modele językowe (ang. *Large Language Models*, LLM). Sam rynek LLM jest jednak bardzo szeroki: jego główną część stanowią trenowanie wielkich modeli językowych o dziesiątkach i setkach miliardów parametrów, budowanie systemów w oparciu o chmurowe interfejsy programistyczne, systemy multimodalne, agenci planujący wieloetapowe zadania oraz komercyjne asystenci. Niniejszy rozdział nie pokrywa tego obszaru w całości. Ograniczono się do wycinka na przecięciu trzech zagadnień: małych modeli językowych (ang. *Small Language Models*, SLM), inferencji na urządzeniu brzegowym oraz interfejsu głosowego.

Kwantyzacja wag, współpraca modeli o różnej wielkości, inżynieria promptów, dobór hiperparametrów oraz łańcuch przetwarzania mowy są rozumiane jako podzbiory wymienionych grup, a nie jako osobne pola badawcze. Schemat ilustrujący ten zakres jest w przygotowaniu i nie wchodzi do tej wersji.

2.1. Małe modele językowe

2.1.1. Modele językowe

Fundamentem współczesnego przetwarzania języka naturalnego (ang. *Natural Language Processing*, NLP) jest architektura Transformer [1]. Na niej oparte są niemal wszystkie najnowsze modele językowe popularnych dostawców.

W ostatnich latach widoczny stał się nurt rozwijania małych modeli językowych (ang. *Small Language Models*, SLM), o rozmiarze rzędu kilku miliardów parametrów. Dzięki treningowi prowadzonemu na wysokiej jakości, selektywnie dobranych danych osiągi tych modeli potrafią zbliżyć się do jednostek wielokrotnie od nich większych. Potwierdzają to raporty techniczne opublikowane przez firmy Meta [2], Microsoft [3] oraz Google [4]. Modele Llama 3, Phi-3 i Gemma 2 osiągają wyniki konkurencyjne względem modeli o rozmiarach rzędu dziesiątek miliardów parametrów, przy znacznie mniejszych wymaganiach sprzętowych. Nie wszystkie modele SLM są jednak oparte na architekturze Transformer. Polski model Bielik [5] oparty jest na architekturze Mistral.

Rynek małych modeli językowych rozwija się dynamicznie. W trakcie realizacji badań będących częścią niniejszej pracy na rynku pojawiły się kolejne generacje modeli znanych dostawców. Okazuje się, że mimo tej samej architektury oraz producenta optymalna konfiguracja może nie być transferowalna na kolejną generację [6].

2.1.2. Wielomodelowe systemy LLM

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

2.1.3. Inżynieria promptów i dostarczanie kontekstu

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

2.1.4. Optymalizacja hiperparametrów generacji

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

2.1.5. Fine tuning modeli językowych

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

2.2. Inferencja na urządzeniu brzegowym

2.2.1. Architektury hybrydowe Edge–Cloud

Modele językowe mogą być uruchamiane lokalnie, na urządzeniu brzegowym (ang. *edge computing*), albo po stronie serwera, w chmurze (ang. *cloud computing*).

Architektura chmurowa oferuje bardzo duże zasoby obliczeniowe, zwykle większe niż urządzenia brzegowe czy serwery prywatne. Wadą tych rozwiązań jest niezerowe opóźnienie komunikacyjne, wynikające z konieczności transmitowania danych. Drugim ograniczeniem obliczeń chmurowych jest zależność od stałego połączenia z internetem. Istotne pozostają również kwestie prywatności: dane użytkownika muszą zostać przesłane do zewnętrznego dostawcy mocy obliczeniowej [6].

Architektura brzegowa umożliwia przetwarzanie lokalne i eliminuje narzut komunikacyjny. Dane pozostają na urządzeniu. Ciągnie to jednak za sobą pewne ograniczenia. Urządzenia brzegowe dysponują zazwyczaj znacznie mniejszą mocą obliczeniową, dlatego stosowane modele muszą być odpowiednio mniejsze. Badania empiryczne pokazują, że procesory ARM w komputerach jednopłytkowych osiągają wydajność rzędu od 5 do 15 tokenów na sekundę dla modeli, których wielkość mieści się w granicach 1,5 miliarda parametrów oraz od 2 do 15 tokenów na sekundę dla modeli posiadających 3 miliardy parametrów [7]. Dla tej klasy sprzętu modele o rozmiarze od 2 do 4 miliardów parametrów stanowią często stosowany zakres kompromisu między jakością a czasem generacji [7].

Wąskim gardłem inferencji (procesu generowania odpowiedzi) na procesorze bez GPU pozostaje przepustowość pamięci (ang. *memory bandwidth*) oraz konieczność utrzymania temperatury poniżej progu przegrzewania [8], [9]. Generowanie kolejnych tokenów wymaga cyklicznego odczytu wag neuronów z pamięci, dlatego im większy model i dłuższy kontekst, tym bardziej skomplikowany jest proces generowania. Z literatury wynika, że na urządzeniach brzegowych skrócenie czasu odpowiedzi wymaga ograniczenia liczby wag (dobór modelu) i długości przetwarzanego kontekstu, a nie wyłącznie zwiększania mocy obliczeniowej.

Kompromisem pomiędzy architekturami brzegową i chmurową jest architektura hybrydowa, w której trudniejsze rozumowanie (ang. *reasoning*) realizowane jest na lokalnym

serwerze, natomiast inferencja obsługująca użytkownika odbywa się na urządzeniu brzegowym [6]. Podejście to zachowuje prywatność danych (brak ryzyka związanego z transmisją), ponieważ serwer działa lokalnie, jednocześnie umożliwiając użycie modelu większego niż ten uruchamiany bezpośrednio na brzegu. Modele na lokalnym serwerze nie są jednak zwykle tak rozbudowane jak modele dostępne w chmurze.

Z przeprowadzonego przeglądu wynika, że dla asystenta korzystającego z danych systemu IoT pełne przeniesienie inferencji do chmury jest niekorzystne. Wprowadza zależność od łącza internetowego, bardzo niestabilnego w Wildzie. Czyste przetwarzanie brzegowe z kolei wymusza użycie jedynie małego modelu językowego i ograniczenie długości kontekstu. Modele mniejsze niż około 1,5 miliarda parametrów często nie osiągają zadowalającej jakości odpowiedzi w języku polskim, natomiast modele przekraczające 4 miliardy parametrów wymagają możliwości przekraczających dostępne lokalnie moce obliczeniowe [7]. Kierunkiem przyjętym w dalszej części pracy jest więc architektura hybrydowa: lokalny serwer wspomagający analizę, a urządzenie brzegowe, obsługuje odpowiedź skierowaną do użytkownika.

2.2.2. Kwantyzacja dla inferencji na sprzęcie bez GPU

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

2.3. Interfejs głosowy

2.3.1. Interfejsy głosowe: rozpoznawanie mowy, detekcja aktywności głosowej i synteza mowy

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

3. Koncepcja i architektura systemu

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

3.1. Założenia projektowe

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

3.2. Architektura Nauczyciel–Uczeń

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

3.3. Schemat przepływu danych

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4. Środowisko projektowo-badawcze

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4.1. Wybór platformy brzegowej

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4.2. Architektura sprzętowa

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4.3. Modele LLM w przetwarzaniu brzegowym danych

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4.4. Wykorzystane oprogramowanie i technologie

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4.4.1. Aplikacja użytkownika

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4.4.2. Warstwa serwerowa i komunikacyjna

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4.4.3. Moduły STT i TTS

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4.4.4. LLM

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4.4.5. Moduł kontekstu

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4.4.6. Narzędzia badawcze

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5. Część badawcza: analiza porównawcza i optymalizacja SLM na urządzeniu brzegowym

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.1. Architektura i metodologia systemu testowego

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.1.1. Komponenty systemu testowego

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.1.2. Zbiór danych testowych

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.1.3. Metryki oceny

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.1.4. Dostrajanie parametrów pracy modelu

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.1.5. Porównywane modele

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.2. Etap Optuna

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.2.1. Warunek termiczny pomiaru

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.2.2. Dominacja flagi halucynacji

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.2.3. Opóźnienie E2E i model Gaussa

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.3. Etap kwantyzacji

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.3.1. Typy kwantyzacji GGUF

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.3.2. Wstępne pomiary E2E

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.3.3. Wyniki

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.3.4. Anomalie i wnioski

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.4. Inżynieria promptów i buforowanie prefiksu

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.4.1. Buforowanie prefiksu promptu systemowego

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.4.2. Budowa promptu systemowego

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.5. Rekomendacje produkcyjne

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6. Część implementacyjna: realizacja prototypu

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6.1. Architektura prototypu

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6.2. Elementy systemu i wykorzystane technologie

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6.2.1. Aplikacja użytkownika

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6.2.2. Transkrypcja mowy (STT)

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6.2.3. Moduł kontekstu

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6.2.4. LLM Nauczyciel

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6.2.5. SLM Uczeń

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6.2.6. Synteza mowy (TTS)

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6.2.7. Konfiguracja

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6.3. Testy rozwiązania

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

7. Podsumowanie

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

7.1. Wnioski z przeprowadzonych badań

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

7.2. Ocena prototypu względem założeń

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

7.3. Plan rozwoju

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

Bibliografia

[1] A. Vaswani i in., "Attention is all you need", *Advances in neural information processing systems*, t. 30, 2017, Seminal paper on Transformer architecture.

[2] A. Dubey, A. Jauhri, A. Pandey, A. Keshwam, A. Al-Dahle i in., "The Llama 3 Herd of Models", *Meta Publications*, 2024, Meta AI Technical Report. URL: https://ai.meta.com/research/publications/the-llama-3-herd-of-models/

[3] M. Abdin, S. A. Jacobs, A. A. Awan, J. Bi i in., "Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone", *arXiv preprint arXiv:2404.14219*, 2024, Microsoft Research. URL: https://arxiv.org/abs/2404.14219

[4] G. Team i G. DeepMind, "Gemma 2: Improving Open Language Models at a Practical Size", *Semantic Scholar*, 2024, Google DeepMind Technical Report. URL: https://www.semanticscholar.org/paper/Gemma-2%3A-Improving-Open-Language-Models-at-a-Size-Riviere-Pathak

[5] SpeakLeash Team, *Bielik-7B-v2: Polish Language Model*, https://huggingface.co/speakleash/Bielik-7B-v2, Software, Accessed: 2024-10-15, 2024.

[6] H. Renney, F. Trad, M. Mattarock i Z. Wood, "Cloud to Edge: Benchmarking LLM Inference On Hardware-Accelerated Single-Board Computers", *arXiv preprint arXiv:2604.24785*, 2026, Carnegie Mellon University; University of Exeter. URL: https://arxiv.org/abs/2604.24785

[7] T. Nguyen i in., "An Evaluation of LLMs Inference on Popular Single-board Computers", *arXiv preprint arXiv:2511.07425*, 2025, University of Technology Sydney. URL: https://arxiv.org/abs/2511.07425

[8] S. Kim i in., "SqueezeLLM: Dense-and-Sparse Quantization", w: *International Conference on Machine Learning*, UC Berkeley, 2023. URL: https://arxiv.org/abs/2306.07629

[9] P. Tummalapalli, S. Arayakandy, R. Pal i K. Kundan, "LLM Inference at the Edge: Mobile, NPU, and GPU Performance Efficiency Trade-offs Under Sustained Load", *arXiv preprint arXiv:2603.23640*, 2026. URL: https://arxiv.org/abs/2603.23640